If the above steps complete successfully, a small shell script will be installed in *~/.local/bin* to start JHBuild. For the script to run, you need to add *~/.local/bin* to the PATH.

```
PATH = $PATH:~/.local/bin
```

To permanently add *~/.local/bin* to the PATH variable, run the following command:

```
echo 'PATH = $PATH:~/.local/bin' >> ~/.bashrc
```

JHBuild prerequisites

Before any modules can be built, it is necessary to have certain build tools installed. Common build tools include the GNU Autotools (autoconf, automake, libtool and gettext), the GNU toolchain (binutils, gcc, g++), make, pkg-config and Python, depending on which modules will be built. JHBuild can check the tools installed using the *sanitycheck* command:

```
jhbuild sanitycheck
```

If this command displays any messages, please install the required packages from the distribution's repository. A list of package names for different distributions is maintained on the GNOME wiki. Run the *sanitycheck* command again after installing the distribution's packages to ensure the required tools are present.

Using JHBuild

After the set-up is complete, JHBuild can be used to build software. To build all the modules selected in the *~/.config/jhbuildrc* file, run the following command:

```
jhbuild build
```

JHBuild will download, configure, compile and install each of the modules. If an error occurs at any stage, JHBuild will present a menu that asks you what needs to be done. The choices include dropping to a shell to fix the error, rerunning the build from various stages, giving up on the module, or ignoring the error and continuing.

Note: Giving up a module will cause other modules that depend on it to fail.

Shown below is an example of the menu displayed when you get an error message:

```
[1] Return phase build
[2] Ignore error and continue to install
[3] Give up on module
[4] Start shell
[5] Reload configuration
[6] Go to phase "wipe directory and start over"
[7] Go to phase "configure"
[8] Go to phase "clean"
[9] Go to phase "distclean"
choice:
```

It is also possible to build a different set of modules and their dependencies by passing the module name as arguments to the build command. For example, to build *gtk+*, use:

```
jhbuild build gtk+
```

If JHBuild is cancelled half way through a build, it is possible to resume the build at a particular module using the *--start-at* option:

```
jhbuild build --start-at = module-name
```

To build one or more modules, ignoring their dependencies, JHBuild provides the *buildone* command. For the *buildone* command to complete successfully, all the dependencies must be previously built and installed or provided by the distribution package.

```
jhbuild buildone gtk+
```

When actively developing a module, you are likely to do so in a source working directory. The 'make' will invoke the build system and install the module. This will be a key part of the edit-compile-install-test cycle.

```
jhbuild make
```

If JHBuild is cancelled half way through a build, it is possible to resume the build at a particular module using the *--start-at* option. To get a list of the modules and dependencies JHBuild will build and the order in which they will be built, use the *list* command:

```
jhbuild list
```

To get information about a particular module, use the *info* command:

```
jhbuild info module-name
```

To download or update all the software sources without building, use the *update* command, which provides an opportunity to modify the sources before building, and can be useful if Internet bandwidth varies:

```
jhbuild update
```

Later, JHBuild can build everything without downloading or updating the sources:

```
jhbuild build --no-network
```

To run a particular command with the same environment